

Make a Note of That

How Better SOPs Supercharge CX Agent
Development

There is a particular argument against detailed SOPs that I have never found especially convincing: we can't possibly document every step.

Of course we can't.

But I don't think anyone seriously advocating for comprehensive documentation is suggesting that we should.

No knowledge base can anticipate every unusual customer, system failure, edge case, or judgment call an employee will encounter. Nor should it try. There will always be situations where the correct answer depends on context, experience, or information nobody has encountered before.

The more interesting question is what happens when organizations fail to document things they do know—and then mistake employees' eventual acquisition of that information for professional development.

I have spent much of my career working in customer experience organizations where the distance between a Tier 1 associate and an SME can appear enormous. Sometimes that distance represents exactly what we assume it does: experience, judgment, pattern recognition, and the ability to navigate ambiguity.

Sometimes the SME just knows where Susan keeps the spreadsheet.

Those things should not be confused.

A mature knowledge system can help separate them. Comprehensive, searchable SOPs give junior employees access to the accumulated procedural knowledge of the people who came before them. That doesn't make those employees experts. It does something I think is almost as important: it allows them to reach the parts of the work where expertise is actually developed much sooner.

The Experience Tax

Every employee has to learn through experience.

There is no SOP that can teach someone exactly how to calm an angry customer, recognize a subtle pattern across several cases, weigh competing risks, or know when something that technically follows policy is nevertheless producing a bad outcome. Those abilities develop through exposure, coaching, feedback, reflection, mistakes, and increasingly difficult work.

But employees also spend an enormous amount of time learning things that someone else already knows.

Which system do I check first?

Does this status mean what it appears to mean?

Is there an exception to this policy?

Who owns this part of the process?

Do I need approval before I do this?

Why does the SOP tell me to do A and then mysteriously jump to D?

In an organization with incomplete documentation, employees acquire these answers through a kind of institutional osmosis. They search old Slack threads (or Teams threads, if you're one of those companies). They ask whoever happens to be available. They listen to more experienced coworkers. They make mistakes. They remember what happened last time.

Eventually, they know.

We often call the resulting difference experience.

I think some of it is better understood as an experience tax.

The organization has already paid to learn these things once. But because that learning was never adequately captured, every new employee has to pay for it again.

There may be learning in that process, but the existence of learning does not mean the process was necessary for learning.

If an associate makes an incorrect account adjustment because nobody documented a prerequisite check, they will probably remember that prerequisite forever. They have learned something.

But if the organization already knew about the prerequisite, what exactly was gained by requiring another person to discover it through failure?

We should be careful about treating every obstacle employees survive as a developmental opportunity.

Some obstacles are just obstacles.

When Informational Privilege Masquerades as Expertise

This becomes particularly important when we evaluate who is ready for more difficult work.

Imagine two employees looking at the same case.

One has been with the company for three years. The other has been there for six months.

The experienced employee knows that one field in the CRM is occasionally unreliable. They know that another team changed its process last year. They remember an exception that was discussed in a meeting. They know that an apparently routine request should be escalated under one particular condition because they once saw what happened when it wasn't.

The newer employee doesn't know any of this.

The experienced employee reaches the correct conclusion almost immediately. The newer employee asks for help.

From the outside, that can look like a straightforward difference in capability.

But several different things are happening at once.

The experienced employee may genuinely have better judgment. They probably recognize patterns faster. They may understand the customer's problem more holistically.

They also possess information the other employee was never given.

When those differences are bundled together, organizations can accidentally create informational privilege masquerading as expertise.

This isn't necessarily because senior employees are hoarding knowledge. Quite the opposite: some of the most experienced people I've worked with have been extraordinarily generous with it.

That's often how the problem becomes visible.

The same SME answers the same question for the fifth time.

Then the tenth.

Then the fiftieth.

At some point, this stops being mentorship. We've turned one of our most knowledgeable employees into a human search engine.

And we've created an odd hierarchy in which access to basic organizational knowledge depends partly on tenure, memory, relationships, and knowing whom to ask.

A good knowledge system strips away some of that artificial distance.

Give both employees access to everything the organization can reasonably externalize and they will not suddenly become equally capable.

But now I can see much more clearly what each of them actually does with the information.

That's where things get interesting.

The Strange Idea That Incomplete Instructions Teach People to Think

One reason organizations sometimes resist highly detailed SOPs is the fear that too much documentation will discourage critical thinking.

I understand the concern.

Nobody wants an associate who can execute a 27-step workflow flawlessly but becomes helpless the moment reality produces Step 28.

A knowledge base can absolutely be designed badly. Processes can become so rigid that employees are rewarded for compliance rather than understanding. Documentation can tell people what to do without helping them understand why. Managers can inadvertently train people to distrust their own observations whenever reality conflicts with the procedure.

But I don't think the solution is to leave established information out.

Because difficulty retrieving known information is not the same thing as difficulty reasoning about information.

Suppose a process contains eight established steps, but the SOP contains six.

The employee reaches Step 6 and gets stuck.

Maybe they search Slack. Maybe they ask an SME. Maybe they make an educated guess. Maybe they get it wrong, receive coaching, and never make that mistake again.

Eventually, they discover Steps 7 and 8.

What critical-thinking ability have we developed?

We've given them a trivia question.

If the organization already knows that Steps 7 and 8 exist, documenting them doesn't rob the employee of an opportunity to think. It gets them through the part of the problem that doesn't require original thought.

Now suppose the employee completes all eight steps and notices that something still doesn't make sense.

The customer's circumstances don't match any documented exception. Two systems contradict each other. The prescribed resolution would technically solve the immediate issue but appears likely to create another one.

Now we have something worth thinking about.

The associate has to interpret evidence. They have to understand why the process exists. They have to identify which assumption has failed. They

may need to weigh risk, propose an alternative, or recognize that the decision exceeds their authority.

Comprehensive documentation hasn't prevented critical thinking.

It has allowed the employee to reach critical thinking sooner.

That's a very different way of thinking about SOPs.

The purpose isn't to do the employee's thinking for them. It's to avoid wasting their thinking on problems the organization has already solved.

Experts Are Terrible at Remembering What Beginners Don't Know

There is another reason documentation becomes incomplete even when nobody intends it to be.

Expertise is strangely good at making itself invisible.

Perform the same workflow hundreds of times and individual decisions begin collapsing into one another. A senior associate says, "I reviewed the account," without consciously registering that reviewing the account actually involves six checks, two conditional decisions, and one subtle warning sign they learned to recognize eighteen months ago.

Ask that person to document the process and they may sincerely believe they've captured everything.

A newer employee follows the instructions and immediately encounters a hole.

This is one reason I think junior employees can be enormously valuable contributors to knowledge management. The expert can tell you whether an SOP is correct.

The beginner can tell you whether it is complete.

Their questions expose invisible assumptions.

"How would I know that?"

"Where does it say to check this?"

"What tells me which option to choose?"

"You said this situation is different. How would I recognize that difference if you weren't sitting next to me?"

Those aren't annoying beginner questions.

They're a knowledge audit.

The answer shouldn't always be another paragraph in the SOP. Some things genuinely require foundational training or judgment. But if the answer is simply an established piece of organizational knowledge that everybody eventually learns, I want to ask a very simple question:

Why are we making everybody eventually learn it?

Write it down.

What “Comprehensive” Actually Means

This is where we can't document every step contains an important truth.

We shouldn't.

Comprehensive documentation doesn't mean maximal documentation.

It means informational completeness proportional to risk.

The standard I would use is:

Could a reasonably capable employee execute this established process safely without relying on undocumented institutional knowledge?

For a low-risk, reversible task, that may require relatively little detail.

For something involving customer money, privacy, contractual commitments, regulatory requirements, irreversible account changes, or meaningful reputational risk, I want considerably stronger guardrails.

I want the employee to understand what they're trying to accomplish, which checks are mandatory, what common exceptions look like, where they have discretion, what they are not authorized to do, and when they need to stop.

Where the rationale matters, I want them to know the rationale too.

That last piece matters because understanding why a step exists is part of what eventually allows someone to recognize when it no longer fits.

An instruction says:

Do this.

A useful SOP can sometimes say:

Do this because of this.

Now the employee has part of the mental model behind the procedure.

Eventually, they may encounter a situation where the underlying condition is different.

And now they have something to reason from.

Safe Autonomy

This is the outcome I'm ultimately interested in.

I don't want employees permanently tethered to documentation. I also don't want to manufacture independence by throwing them into preventable ambiguity and seeing who survives.

I want safe autonomy.

For me, safe autonomy means an employee can independently operate within a well-understood scope, recognize when they've reached its boundary, and seek help before uncertainty becomes preventable harm.

Documentation can make those boundaries visible.

And as employees develop, those boundaries should move.

At first, someone follows the process.

Then they understand the process.

Then they can apply it confidently to variations.

Eventually, they begin recognizing exceptions.

Then they start questioning assumptions.

And one day they come to me and say:

"This keeps happening. I don't think the SOP is the problem. I think the process itself is."

That's an employee who has moved beyond procedural competence.

They aren't simply consuming organizational knowledge anymore.

They're producing it.

And that's where knowledge management begins feeding itself.

Someone encounters a new problem. They work through it with a coach or SME. The organization learns something. The reusable part of that learning gets captured. The next person encounters the same situation with better information.

That person can start further ahead.

Let People Struggle With Better Problems

None of this means an SOP can manufacture expertise.

It can't.

You can give a new associate every document an experienced colleague has ever read and they will still lack the thousands of interactions, mistakes, successes, conversations, and difficult decisions that shaped that colleague's judgment.

Some distance can only be traveled through experience.

But I think leaders should be much more deliberate about which distance we require employees to travel for themselves.

I don't want a Tier 1 associate spending six months learning things we could have told them on day one.

I want them spending those six months applying that knowledge, seeing where it breaks, learning how customers behave, recognizing patterns, encountering exceptions, receiving coaching, making increasingly sophisticated decisions, and eventually finding problems I don't already know how to solve.

Let them struggle.

Just let them struggle with better problems.

Because the purpose of comprehensive documentation isn't to eliminate the learning curve.

It's to make sure employees are climbing a new part of it.

When one employee discovers something the organization didn't know, make a note of that.

The next employee shouldn't have to discover it again.